NithinS_221IT085

October 9, 2025

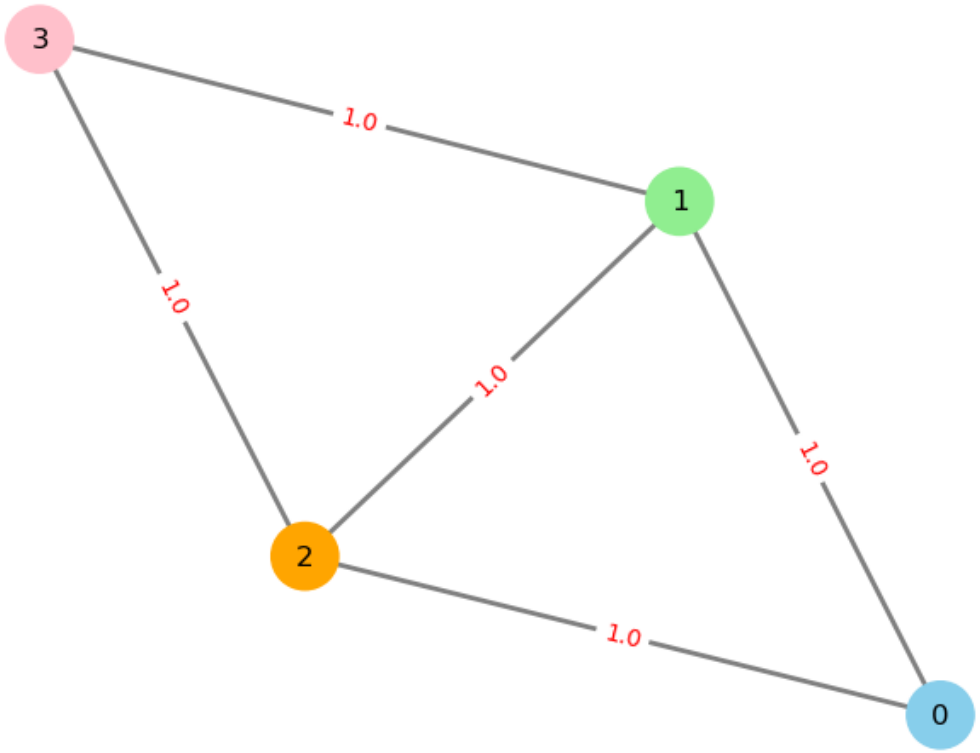
1 Implementing Min Max Cut using QAOA Algorithm

[9]:
```
!pip install networkx
import numpy as np
import networkx as nx

num_nodes = 4
w = np.array(
    [[0.0, 1.0, 1.0, 0.0], [1.0, 0.0, 1.0, 1.0], [1.0, 1.0, 0.0, 1.0], [0.0, 1.
 ↪0, 1.0, 0.0]]
)
G = nx.from_numpy_array(w)
```

```
Requirement already satisfied: networkx in
/home/nithin/miniconda3/envs/ml/lib/python3.12/site-packages (3.5)
```

[10]:
```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

num_nodes = 4
w = np.array(
    [
        [0.0, 1.0, 1.0, 0.0],
        [1.0, 0.0, 1.0, 1.0],
        [1.0, 1.0, 0.0, 1.0],
        [0.0, 1.0, 1.0, 0.0]
    ]
)
G = nx.from_numpy_array(w)

layout = nx.spring_layout(G, seed=10)

colors = ["skyblue", "lightgreen", "orange", "pink"]

nx.draw(
    G,
    pos=layout,
```

```
    with_labels=True,
    node_color=colors,
    node_size=800,
    edge_color="gray",
    width=2
)

labels = nx.get_edge_attributes(G, "weight")
nx.draw_networkx_edge_labels(G, pos=layout, edge_labels=labels, font_color="red")

plt.show()
```



[ ]:
```
import numpy as np

def objective_value(x, w):
    X = np.outer(x, 1 - x)
    w_01 = (w != 0).astype(int)
    return np.sum(w_01 * X)

def bitfield(n, length):
```

```
    return [int(d) for d in np.binary_repr(n, length)]

def brute_force_min_objective(w):
    L = w.shape[0]
    max_val = 2 ** L
    best_obj = np.inf
    for i in range(max_val):
        cur = np.array(bitfield(i, L))
        if np.count_nonzero(cur) * 2 != L:
            continue
        obj_val = objective_value(cur, w)
        if obj_val < best_obj:
            best_obj = obj_val
    return best_obj

num_nodes = 4
w = np.array([
    [0.0, 1.0, 1.0, 0.0],
    [1.0, 0.0, 1.0, 1.0],
    [1.0, 1.0, 0.0, 1.0],
    [0.0, 1.0, 1.0, 0.0]
])

sol = brute_force_min_objective(w)
print(f"Objective value computed by brute-force method is {sol}")
```

```
Objective value computed by the brute-force method is 3
```

[ ]:
```
from qiskit.quantum_info import Pauli, SparsePauliOp
import numpy as np

def generate_edge_paulis(weight_matrix):
    num_nodes = len(weight_matrix)
    paulis = []
    coeffs = []
    shift = 0

    for i in range(num_nodes):
        for j in range(i):
            if weight_matrix[i, j] != 0:
                z = np.zeros(num_nodes, dtype=bool)
                x = np.zeros(num_nodes, dtype=bool)
                z[i] = True
                z[j] = True
                paulis.append(Pauli((z, x)))
                coeffs.append(-0.5)
                shift += 0.5
```

```
    return paulis, coeffs, shift

def generate_full_paulis(num_nodes):
    paulis = []
    coeffs = []
    shift = 0

    for i in range(num_nodes):
        for j in range(num_nodes):
            z = np.zeros(num_nodes, dtype=bool)
            x = np.zeros(num_nodes, dtype=bool)
            z[i] = True
            if i != j:
                z[j] = True
                paulis.append(Pauli((z, x)))
                coeffs.append(1.0)
            else:
                shift += 1
    return paulis, coeffs, shift

def get_operator(weight_matrix):
    edge_paulis, edge_coeffs, edge_shift = generate_edge_paulis(weight_matrix)
    full_paulis, full_coeffs, full_shift =␣
 ↪generate_full_paulis(len(weight_matrix))

    paulis = edge_paulis + full_paulis
    coeffs = edge_coeffs + full_coeffs
    shift = edge_shift + full_shift

    return SparsePauliOp(paulis, coeffs=coeffs), shift

qubit_op, offset = get_operator(w)
```

So lets use the QAOA algorithm to find the solution.

[ ]:
```
from qiskit.primitives import StatevectorSampler
from qiskit.quantum_info import Pauli
from qiskit.result import QuasiDistribution

from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA

from qiskit_algorithms.utils import algorithm_globals

sampler = StatevectorSampler(seed=42)
```

```
def sample_most_likely(state_vector):
    return np.array([int(bit) for bit in max(state_vector.items(), key=lambda x:␣
 ↪x[1])[0][::-1]])


algorithm_globals.random_seed = 10598

optimizer = COBYLA()
qaoa = QAOA(sampler, optimizer, reps=2)

result = qaoa.compute_minimum_eigenvalue(qubit_op)

x = sample_most_likely(result.eigenstate)

print(x)
print(f"Objective value computed by QAOA is {objective_value(x, w)}")
```

```
[1 1 0 0]
Objective value computed by QAOA is 3
```

[ ]:
```
from qiskit_algorithms import NumPyMinimumEigensolver
from qiskit.quantum_info import Operator

npme = NumPyMinimumEigensolver()
result = npme.compute_minimum_eigenvalue(Operator(qubit_op))

x = sample_most_likely(result.eigenstate.probabilities_dict())

print(x)
print(f"Objective value computed by the NumPyMinimumEigensolver is␣
 ↪{objective_value(x, w)}")
```

```
[1 1 0 0]
Objective value computed by the NumPyMinimumEigensolver is 3
```

[ ]:
```
from qiskit.circuit.library import n_local

from qiskit_algorithms import SamplingVQE
from qiskit_algorithms.utils import algorithm_globals

algorithm_globals.random_seed = 13345

optimizer = COBYLA()
ansatz = n_local(qubit_op.num_qubits, "ry", "cz", reps=2, entanglement="linear")
sampling_vqe = SamplingVQE(sampler, ansatz, optimizer)

result = sampling_vqe.compute_minimum_eigenvalue(qubit_op)
```

```
x = sample_most_likely(result.eigenstate)

print(x)
print(f"Objective value computed by SamplingVQE is {objective_value(x, w)}")
```

```
[0 1 0 1]
Objective value computed by SamplingVQE is 3
```

[ ]:
```
from qiskit_aer.noise import NoiseModel
from qiskit.providers.fake_provider import GenericBackendV2

coupling_map = [(0, 1), (1, 2), (2, 3)]
backend = GenericBackendV2(num_qubits=4, coupling_map=coupling_map, seed=54)
```

Let us define a PassManager for this backend.

[ ]:
```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

pm = generate_preset_pass_manager(optimization_level=2, backend=backend)
```

[ ]:
```
def callback(**kwargs):
    if kwargs["count"] == 0:
        print(f"Callback function has been called!")
```

[ ]:
```
qaoa = QAOA(sampler, optimizer, reps=2, transpiler=pm,␣
 ↪transpiler_options={"callback": callback})
result = qaoa.compute_minimum_eigenvalue(qubit_op)

x = sample_most_likely(result.eigenstate)

print(x)
print(f"Objective value computed by QAOA is {objective_value(x, w)}")
```

```
Callback function has been called!
[0 1 0 1]
Objective value computed by QAOA is 3
```

[ ]: